

Understanding Complexity Classes: P, NP, and Beyond

Exploring P, NP, co-NP, NP-Hard, and NP-Complete

What is a Complexity Class?

- A set of problems grouped by resource requirements
 - Time or space needed to solve
- Helps compare difficulty of computational problems
- Key to understanding what computers can and cannot do



The Class P: Polynomial Time

P is the set of decision problems that can be solved efficiently (in polynomial time) by a deterministic Turing machine. These are considered 'easy' or 'tractable' problems in computer science.



DFA vs Turing Machine

Finite Automata

- Finite number of states
- No memory beyond states
- Accepts regular languages
- Cannot perform general computation

Turing Machine

- Infinite tape for memory
- Can move left and right
- Accepts more complex languages
- Can simulate any algorithm

DFA and Turing Machine (simple problems)

$L = \{w = \{0,1\}^* \mid w \text{ has even number of } 1\}$

$L = \{w = \{0,1\}^* \mid w \text{ ends with } 01\}$

$L = \{w = \{0^n, 1^n\} \mid n \geq 0\}$

Examples of Problems in P

- Sorting a list of numbers (e.g., Merge Sort)
- Finding the shortest path in a graph (Dijkstra's Algorithm)
- Multiplying two numbers



The Class NP: Nondeterministic Polynomial Time

NP is the set of decision problems for which a solution can be verified in polynomial time by a deterministic Turing machine. It is not required that the solution can be found quickly, only that it can be checked quickly.



Examples of NP Problems

- Sudoku puzzle solutions
 - Easy to check, hard to solve
- Traveling Salesman Problem (decision form)
- Boolean Satisfiability (SAT)





Comparing P and NP

P

Problems can be solved and verified quickly
(polynomial time).

NP

Solutions can be verified quickly, but may not be
found quickly.

**All problems in P are also in NP, but
not all NP problems are known to
be in P.**

The P vs NP Question

The biggest open question in computer science: Is every problem whose solution can be quickly verified (NP) also quickly solvable (P)? No one knows the answer yet!

It is one of the seven “Millenium Prize Problems”, presented by Clay Mathematics Institute in 2000. One problem is solved so far from this list.



Other Complexity Classes

co-NP

Problems whose 'no' answers can be verified quickly.

NP-Hard

At least as hard as the hardest problems in NP.

NP-Complete

Problems in NP that are as hard as any in NP.

Relationship

$\text{NP-Complete} = \text{NP} + \text{NP-Hard}$

NP-Hard Problem

A problem X is NP-hard if every problem in NP can be polynomial-time reduced to X . The problem may or may not be in NP.

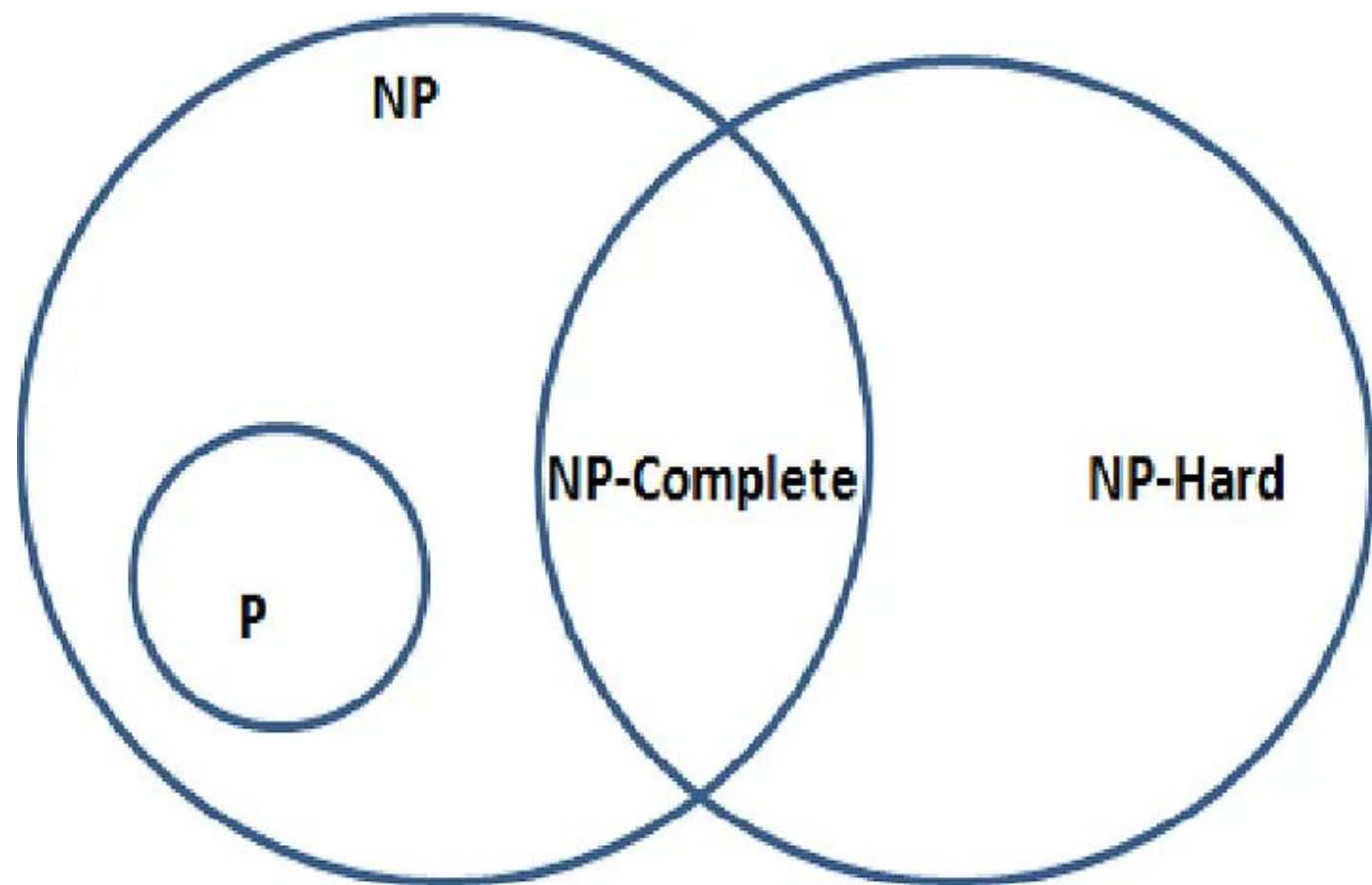
- Halting Problem (Not in NP, because it is not solvable at all)

- TSP (optimization version) [Not in NP, because “verifying optimality” requires showing no shorter tour exists, which is not polynomially checkable.]
- Generalized chess/checker/go (Can verify in exponential time)

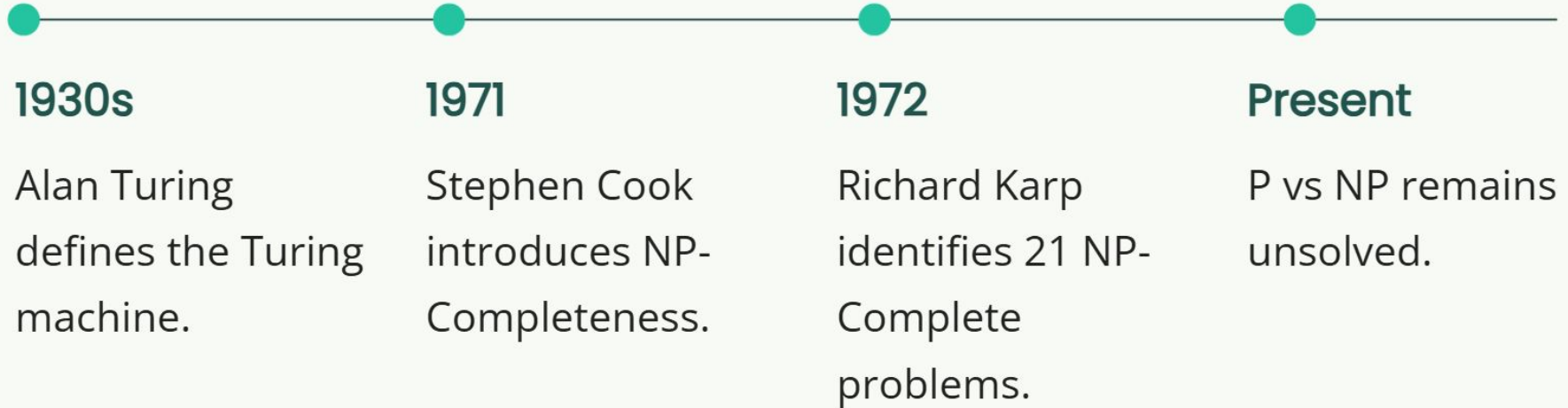
These type of NP-hard problems are categorized as exptime problems.

Summary of Key Complexity Classes

Class	Definition	Example
P	Solvable in polynomial time	Sorting
NP	Verifiable in polynomial time	Sudoku
co-NP	'No' answers verifiable in polynomial time	TAUTOLOGY
NP-Complete	Hardest in NP	SAT
NP-Hard	At least as hard as NP-Complete	Halting Problem



History of Complexity Theory



Undecidable Problem

Halting Problem is an undecidable problem.

What is the Halting Problem?

- Asks if a program will halt or loop forever
- Input: a program and its input
- Output: True (halts) or False (loops forever)



Proof by Contradiction: The Approach

- To prove the Halting Problem is undecidable, we assume there is a program, $\text{Halt}(P, I)$, that can always decide if P halts on input I .
- We then construct a scenario that leads to a logical contradiction.
- We construct a new program D , with the following description:



Description of the program D

- D can take any program P as input.
- D calls the hypothetical Halt function with P as both the program and the input $\text{Halt}(P, P)$ to be analyzed.
- If $\text{Halt}(P, P)$ returns True, D runs an infinite loop.
- If $\text{Halt}(P, P)$ returns False, D will immediately halt and finish executing.

Now if we call $D(D)$, what will happen?



The Contradiction: Running D on Itself

Case 1: $D(D)$ Halts

$\text{Halt}(D, D)$ would return True, so D should loop forever. Contradicts halting.

Case 2: $D(D)$ Loops Forever

$\text{Halt}(D, D)$ would return False, so D should halt. Contradicts looping.